

NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

Stacks and Queues – Tutorials Answers

Dr. Owen Noel Newton Fernando

College of Engineering

School of Computer Science and Engineering



Q01 – reverse_stack(stack)

(reverseStack)

Write a Python function reverse_stack() that reverses a stack using a queue.

The reverse_stack() function only uses push() and pop() when adding or removing integers from the stack, and only uses enqueue() and dequeue() when adding or removing integers from the queue.

Function prototype

```
def reverse_stack(stack):
```

Goal: reverse the order of all stack elements using one queue.

Content Copyright Nanyang Technological University

Q01 – Algorithm Idea

1. Pop all elements from the stack and enqueue them into a queue.

2. Dequeue all elements from the queue and push them back to the stack.

3. If the stack is empty, there is nothing to reverse.

Stack = LIFO • Queue = FIFO

Content Copyright Nanyang Technological University

Q01 – Python Function

```
def reverse_stack(stack):
    if stack.isEmpty():
        return

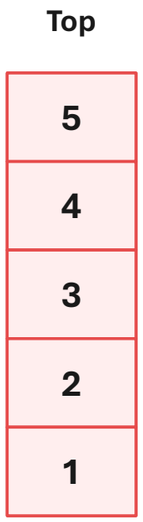
    queue = Queue()
    # Step 1: Pop from stack and enqueue
    while not stack.isEmpty():
        queue.enqueue(stack.pop())
    # Step 2: Dequeue from queue and push back
    while not queue.isEmpty():
        stack.push(queue.dequeue())
```

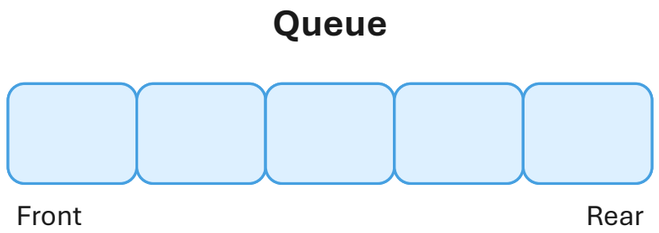
Content Copyright Nanyang Technological University

Q01 – Example Overview

Example stack: 5, 4, 3, 2, 1

5 is initially at the TOP of the stack.



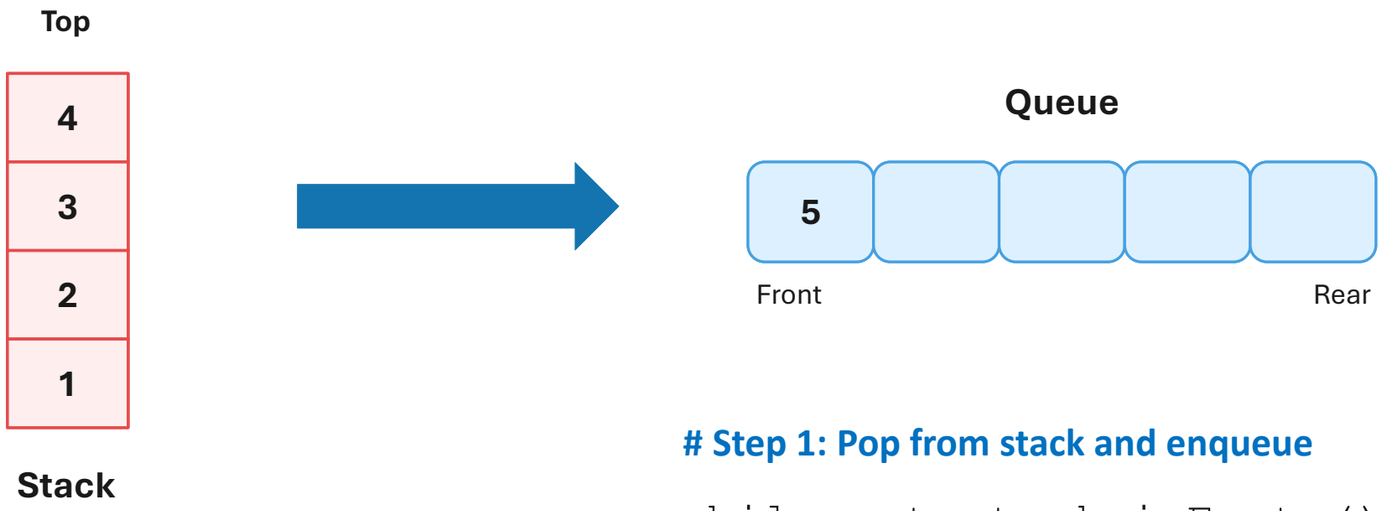



```
queue = Queue()
```

Initial state: Stack = ⟨5,4,3,2,1⟩ (top = 5); Queue is empty.

Content Copyright Nanyang Technological University

Q01 – Step 1.1: Pop 5 → Enqueue 5



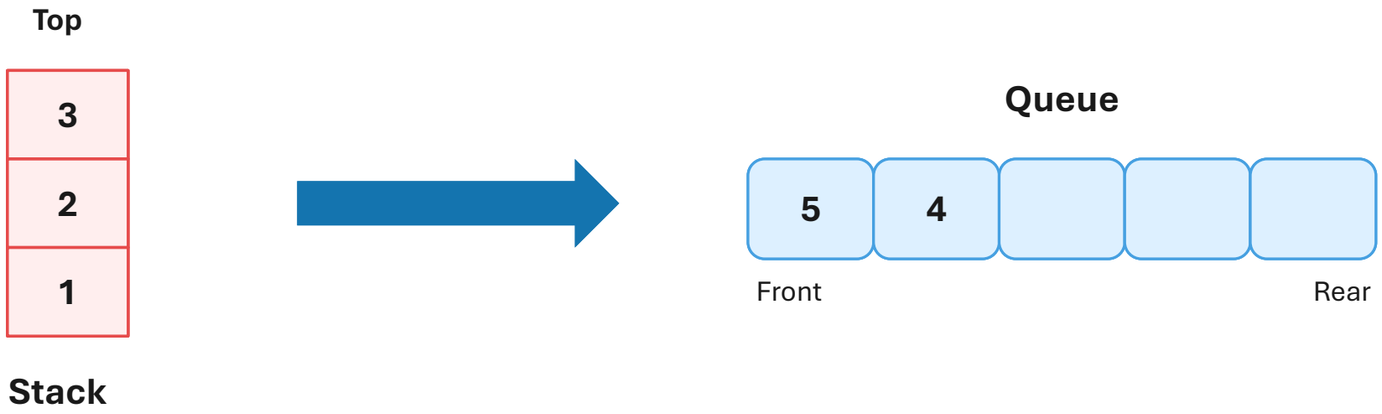
Step 1: Pop from stack and enqueue

```
while not stack.isEmpty():
    queue.enqueue(stack.pop())
```

Pop 5 from the stack and enqueue 5 into the queue.

Content Copyright Nanyang Technological University

Q01 – Step 1.2: Pop 4 → Enqueue 4

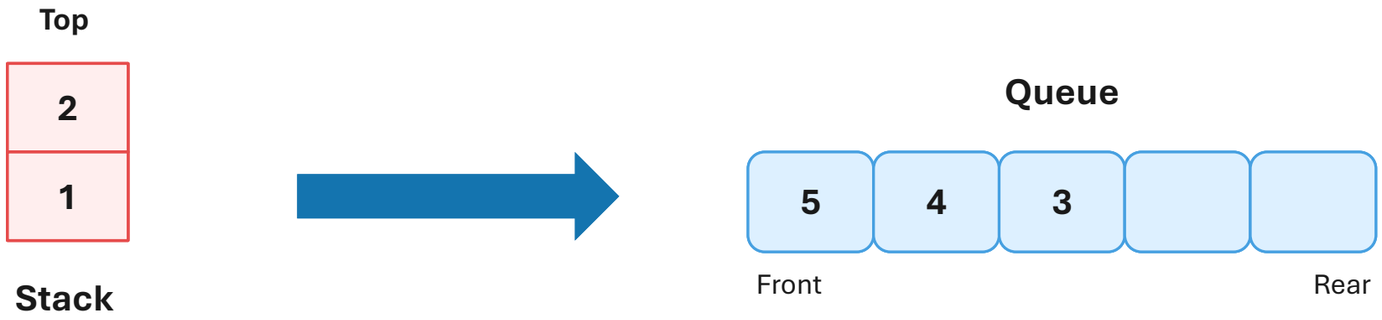



```
# Step 1: Pop from stack and enqueue
while not stack.isEmpty():
    queue.enqueue(stack.pop())
```

Pop 4 from the stack and enqueue 4 into the queue.

Content Copyright Nanyang Technological University

Q01 – Step 1.3: Pop 3 → Enqueue 3



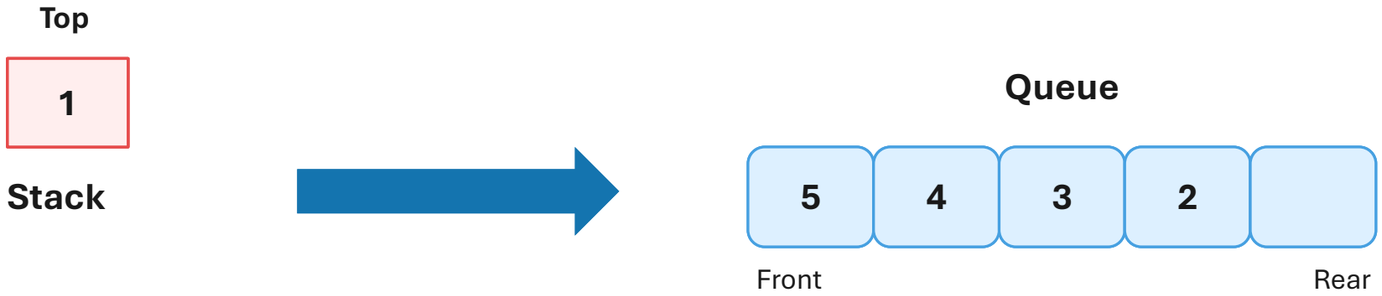
Step 1: Pop from stack and enqueue

```
while not stack.isEmpty():
    queue.enqueue(stack.pop())
```

Pop 3 from the stack and enqueue 3 into the queue.

Content Copyright Nanyang Technological University

Q01 – Step 1.4: Pop 2 → Enqueue 2

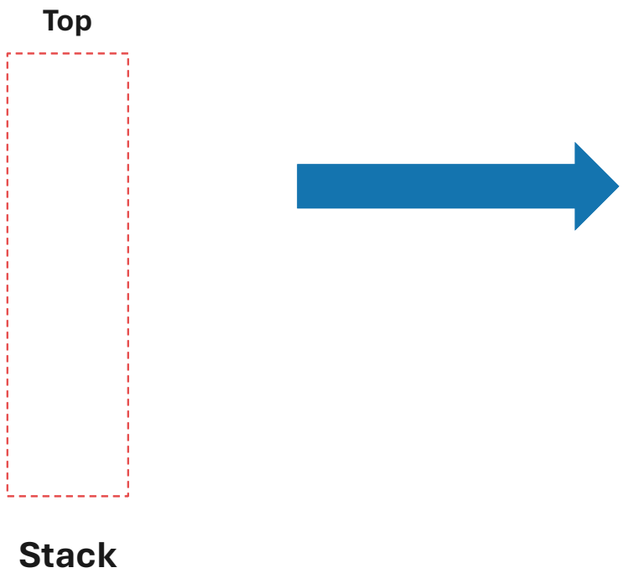


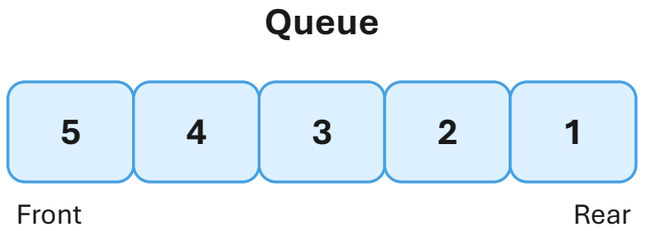

```
# Step 1: Pop from stack and enqueue
while not stack.isEmpty():
    queue.enqueue(stack.pop())
```

Pop 2 from the stack and enqueue 2 into the queue.

Content Copyright Nanyang Technological University

Q01 – Step 1.5: Pop 1 → Enqueue 1





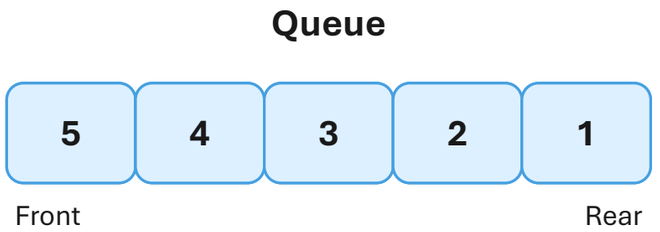

```
# Step 1: Pop from stack and enqueue
while not stack.isEmpty():
    queue.enqueue(stack.pop())
```

Pop 1 from the stack and enqueue 1 into the queue.

Content Copyright Nanyang Technological University

Q01 – Result after Step 1

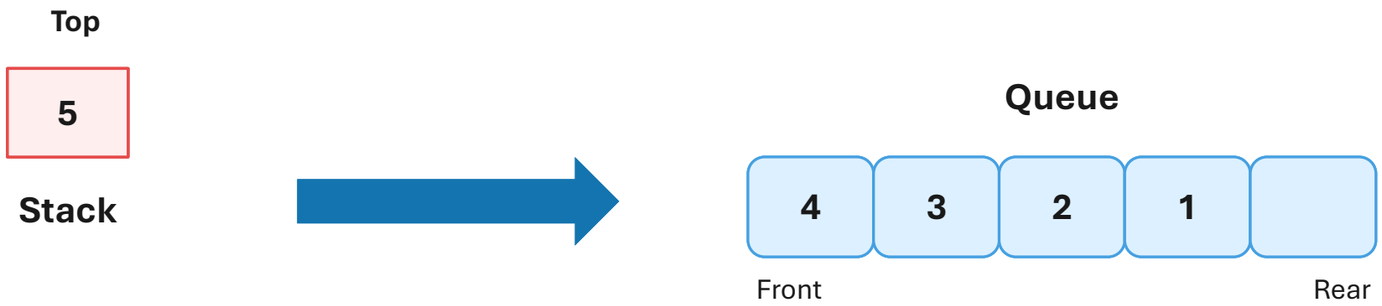




Result: The stack is empty. The queue contains all elements in the original top-to-bottom order, with 5 at the front.

Content Copyright Nanyang Technological University

Q01 – Step 2.1: Dequeue 5 → Push 5

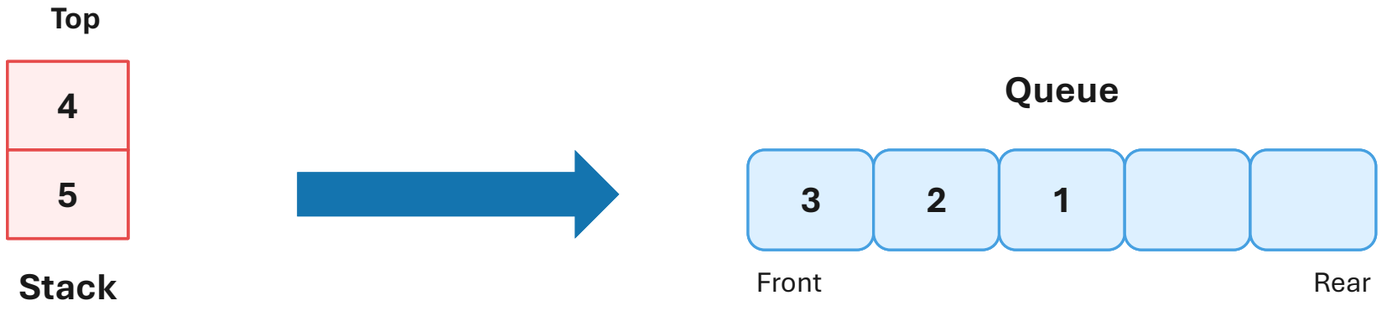



```
# Step 2: Dequeue from queue and push back
while not queue.isEmpty():
    stack.push(queue.dequeue())
```

Dequeue 5 from the queue and push 5 onto the stack.

Content Copyright Nanyang Technological University

Q01 – Step 2.2: Dequeue 4 → Push 4

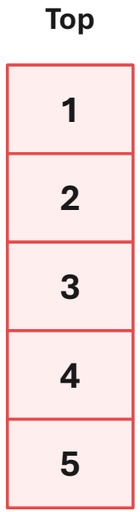



```
# Step 2: Dequeue from queue and push back
while not queue.isEmpty():
    stack.push(queue.dequeue())
```

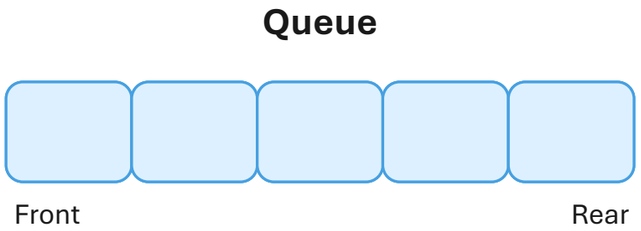
Dequeue 4 from the queue and push 4 onto the stack.

Content Copyright Nanyang Technological University

Q01 – Step 2.3: Dequeue 3, 2, 1 → Push





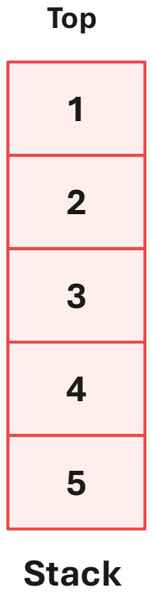


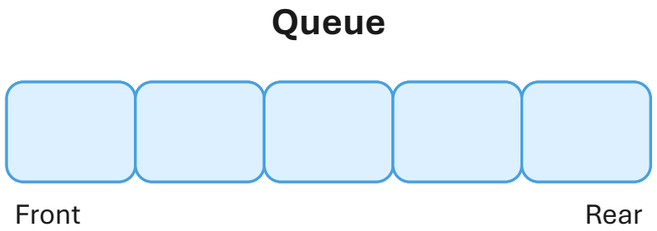

```
# Step 2: Dequeue from queue and push back
while not queue.isEmpty():
    stack.push(queue.dequeue())
```

Continue in FIFO order: dequeue 3, then 2, then 1, and push each value onto the stack.

Content Copyright Nanyang Technological University

Q01 – Final Result





Final Result: Stack = ⟨1,2,3,4,5⟩ with 1 on top — the reverse of the original stack.

Only push()/pop() are used for the stack, and enqueue()/dequeue() for the queue.

Content Copyright Nanyang Technological University

2. Question

(reverseFirstKItems)

Write a Python function reverse_first_k_items() that reverses the order of the first k elements of a queue using a stack, leaving the other elements in the same relative order for a given integer k and a queue of integers.

The function only uses push() and pop() when adding or removing integers from the stack, and only uses enqueue() and dequeue() when adding or removing integers from the queue.

Function prototype:

```
def reverse_first_k_items(queue, k):
```

Content Copyright Nanyang Technological University

2. Algorithm Idea

1. Validate k and the queue.
2. Move the first k queue elements into a stack.
3. Pop all stack elements and enqueue them back to the queue.
4. Rotate the remaining n − k elements from the front to the rear.

Content Copyright Nanyang Technological University

2. Python Function

```
def reverse_first_k_items(queue, k):
    # Validate k and ensure the queue can be processed
    if k <= 0 or queue.is_empty() or k > queue.ll.size:
        return

    # Create a temporary stack
    s = Stack()

    # Step 1: Move the first k queue items to the stack
    for _ in range(k):
        s.push(queue.dequeue())

    # Step 2: Pop from the stack and enqueue back # This reverses the first k items
    while not s.is_empty():
        queue.enqueue(s.pop())

    # Step 3: Move the remaining items from front to rear # This restores them to their original relative order
    for _ in range(queue.ll.size - k):
        queue.enqueue(queue.dequeue())
```

Content Copyright Nanyang Technological University

2. Validation Check

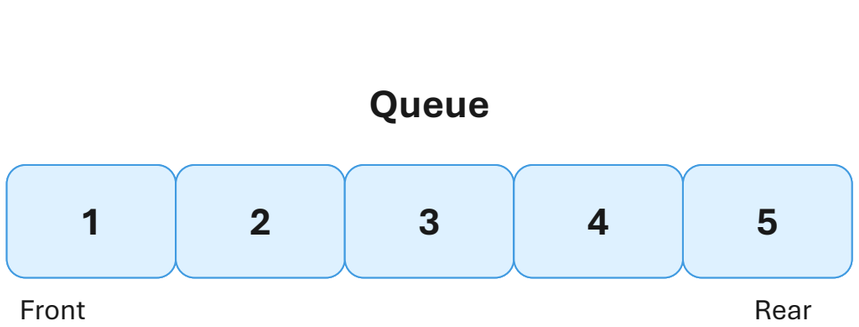
- If k is 0 or negative → nothing to reverse.
- If the queue is empty → nothing to reverse.
- **If k is larger than the queue size → invalid.**

For the walkthrough, Queue = [1, 2, 3, 4, 5] and k = 3.

Content Copyright Nanyang Technological University

2. Example Overview

Queue = [1, 2, 3, 4, 5], k = 3



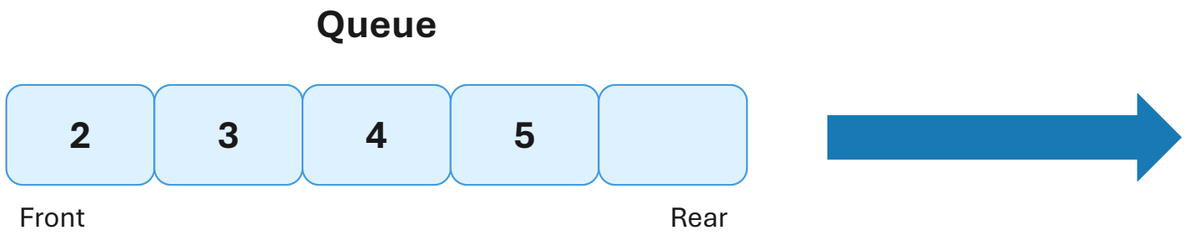



```
# Create a temporary stack
s = Stack()
```

Goal: reverse only the first 3 queue elements. Expected final queue = [3, 2, 1, 4, 5].

Content Copyright Nanyang Technological University

2. Step 1.1 – Dequeue 1 → Push 1

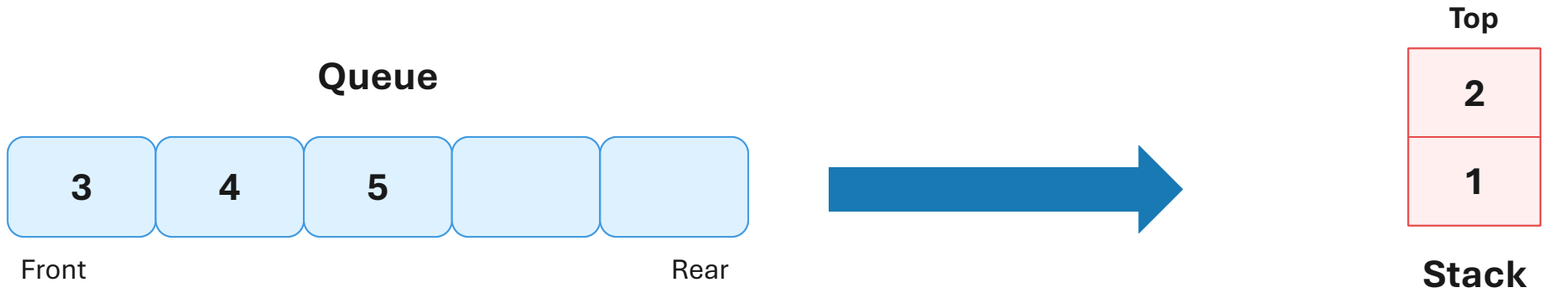



```
# Step 1: Move the first k queue items to the stack
for _ in range(k):
    s.push(queue.dequeue())
```

Dequeue 1 from the front of the queue and push it onto the stack.

Content Copyright Nanyang Technological University

2. Step 1.2 – Dequeue 2 → Push 2

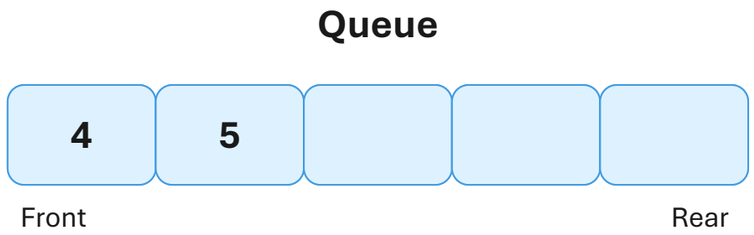



```
# Step 1: Move the first k queue items to the stack
for _ in range(k):
    s.push(queue.dequeue())
```

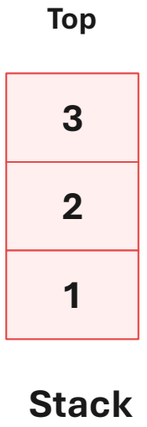
Dequeue 2 and push it onto the stack. Stack top is now 2.

Content Copyright Nanyang Technological University

2. Step 1.3 – Dequeue 3 → Push 3





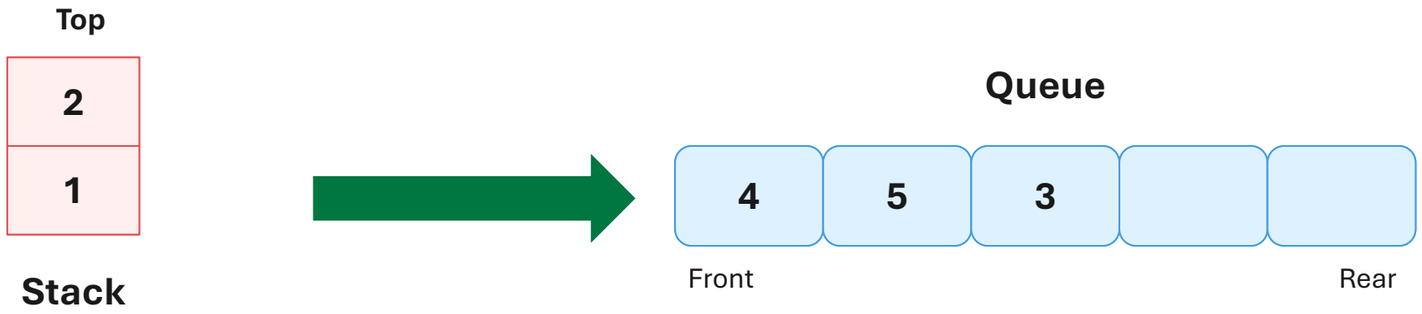



```
# Step 1: Move the first k queue items to the stack
for _ in range(k):
    s.push(queue.dequeue())
```

After k = 3 dequeues, Stack = [3, 2, 1] and Queue = [4, 5].

Content Copyright Nanyang Technological University

2. Step 2.1 – Pop 3 → Enqueue 3

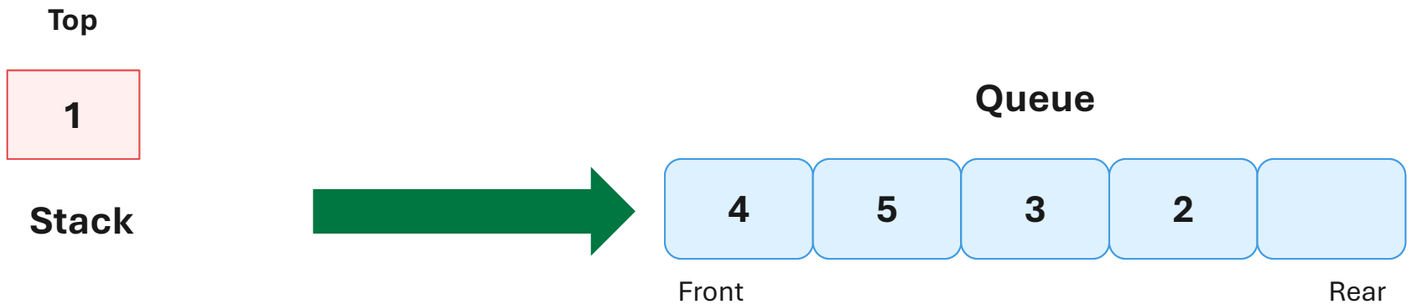



```
# Step 2: Pop from the stack and enqueue back
# This reverses the first k items
while not s.is_empty():
    queue.enqueue(s.pop())
```

Pop 3 from the stack and enqueue it at the rear of the queue.

Content Copyright Nanyang Technological University

2. Step 2.2 – Pop 2 → Enqueue 2

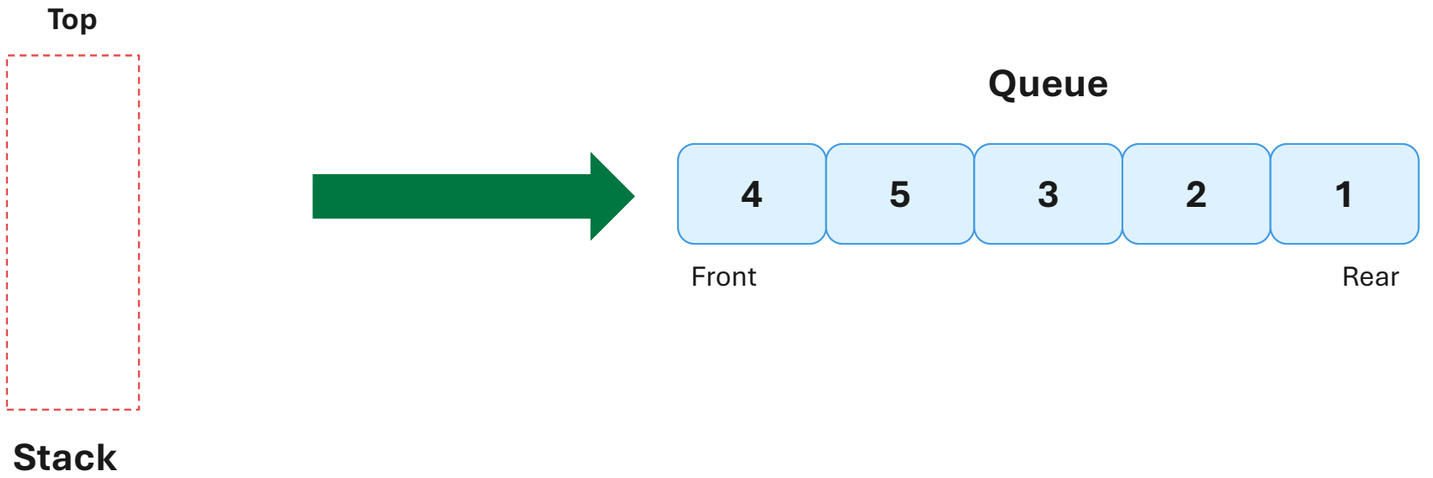



```
# Step 2: Pop from the stack and enqueue back
# This reverses the first k items
while not s.is_empty():
    queue.enqueue(s.pop())
```

Pop 2 from the stack and enqueue it. Queue = [4, 5, 3, 2].

Content Copyright Nanyang Technological University

2. Step 2.3 – Pop 1 → Enqueue 1

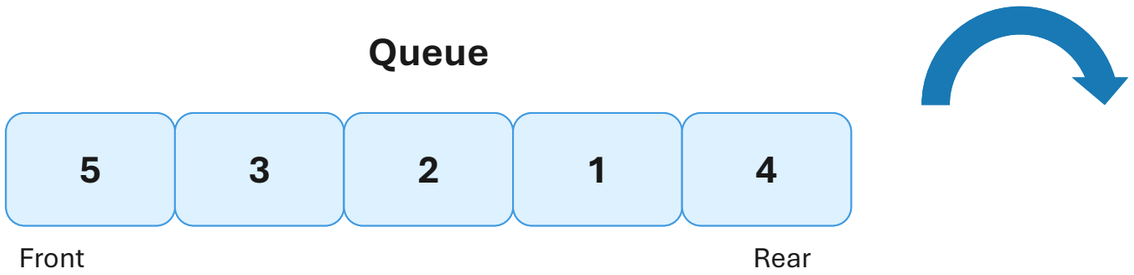



```
# Step 2: Pop from the stack and enqueue back
# This reverses the first k items
while not s.is_empty():
    queue.enqueue(s.pop())
```

Pop 1 and enqueue it. Queue = [4, 5, 3, 2, 1].

Content Copyright Nanyang Technological University

2. Step 3.1 – Rotate Remaining Element 4

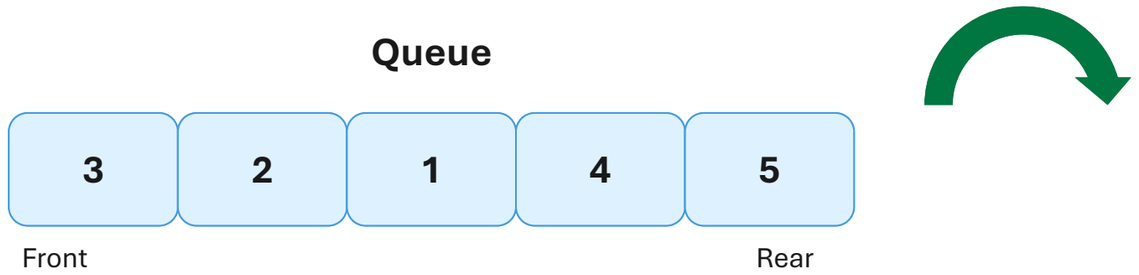



```
# Step 3: Move the remaining items from front to rear # This restores them to their original relative order
for _ in range(queue.ll.size - k):
    queue.enqueue(queue.dequeue())
```

Dequeue 4 from the front and enqueue it at the rear → [5, 3, 2, 1, 4].

Content Copyright Nanyang Technological University

2. Step 3.2 – Rotate Remaining Element 5



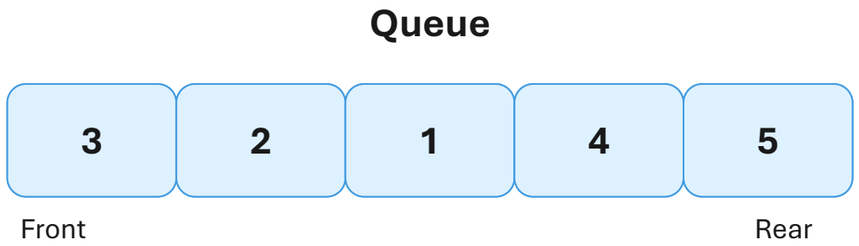

```
# Step 3: Move the remaining items from front to rear # This restores them to their original relative order
for _ in range(queue.ll.size - k):
    queue.enqueue(queue.dequeue())
```

Dequeue 5 from the front and enqueue it at the rear → [3, 2, 1, 4, 5].

Content Copyright Nanyang Technological University

2. Final Result

First k = 3 elements are reversed; the remaining elements keep their relative order.



Final Queue: [3, 2, 1, 4, 5]

Only the first three elements [1, 2, 3] have been reversed to [3, 2, 1].

Content Copyright Nanyang Technological University

3. Question

(sortStack)

Write a Python function sort_stack() that sorts a given stack in ascending order using another temporary stack.

The sort_stack() function only uses push() and pop() when adding or removing integers from the stack.

Function prototype:

```
def sort_stack(stack):
```

Content Copyright Nanyang Technological University

3. Algorithm Idea

1. Create a temporary stack, temp_stack.
2. Pop one item from the original stack into temp.
3. While temp_stack is not empty and its top is greater than temp, move those larger items back to the original stack.
4. Push temp into temp_stack. Repeat until the original stack is empty.
5. Move all items from temp_stack back to the original stack.

Content Copyright Nanyang Technological University

3. Python Function

```
def sort_stack(stack):

    # If the stack is empty, there is nothing to sort
    if stack.is_empty():
        return

    # Create a temporary stack to assist with sorting
    temp_stack = Stack()

    # Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)

    # Move the sorted items back to the original stack
    while not temp_stack.is_empty():
        stack.push(temp_stack.pop())
```

3. Example Overview

Original stack (top → bottom): 3, 1, 4, 2




```
# Create a temporary stack to assist with sorting
    temp_stack = Stack()
```

Goal: sort the stack in ascending order so that the final top-to-bottom order is 1, 2, 3, 4.

Content Copyright Nanyang Technological University

3. Step 1 – Pop 3 → Push to temp_stack

Temp = 3




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

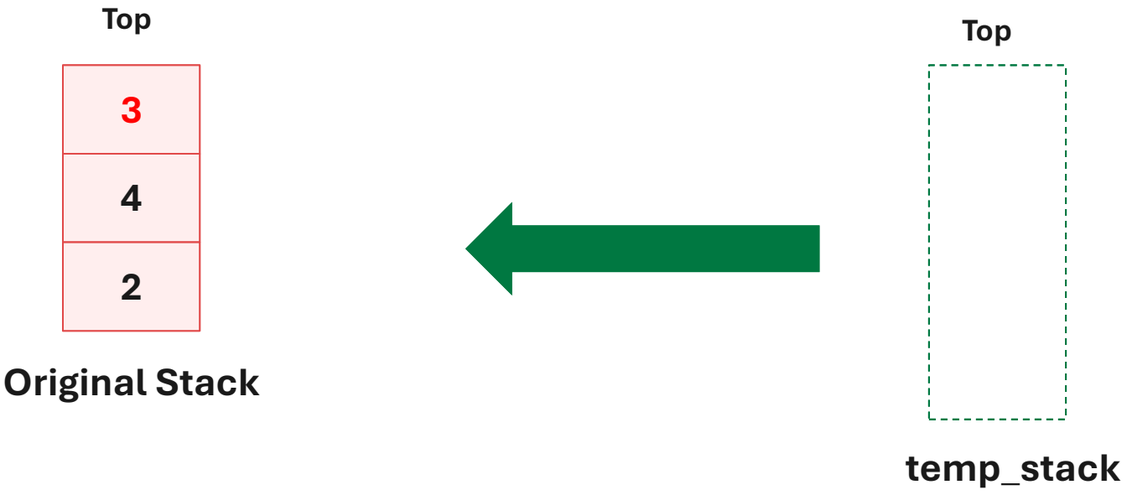
        # Insert temp into its correct position
        temp_stack.push(temp)
```

Pop 3 from the original stack. temp_stack is empty, so push 3 directly.

Content Copyright Nanyang Technological University

3. Step 2 – Pop 1; Move 3 Back

Temp = 1




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

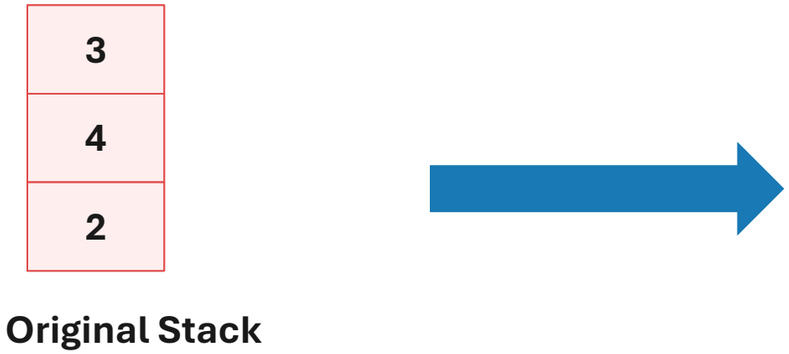
        # Insert temp into its correct position
        temp_stack.push(temp)
```

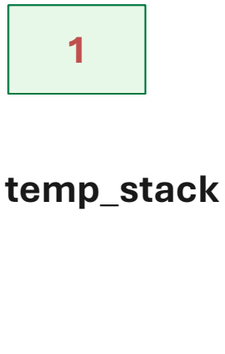
Pop 1. Since temp_stack.peek() = **3 > 1**, pop 3 from temp_stack and push it back to the original stack.

Content Copyright Nanyang Technological University

3. Step 3 – Push 1 to temp_stack

Temp = 1






```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

Now temp_stack is empty, so push 1. Original stack = [3,4,2]; temp_stack = [1].

Content Copyright Nanyang Technological University

3. Step 4 – Pop 3 → Push to temp_stack

Temp = 3




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

Pop 3. Since temp_stack.peek() = 1 is not greater than 3, push 3 onto temp_stack.

Content Copyright Nanyang Technological University

3. Step 5 – Pop 4 → Push to temp_stack

Temp = 4




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

Pop 4. Since temp_stack.peek() = 3 < 4, push 4. temp_stack = [4,3,1].

Content Copyright Nanyang Technological University

3. Step 6 – Pop 2; Move 4 Back

Temp = 2




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

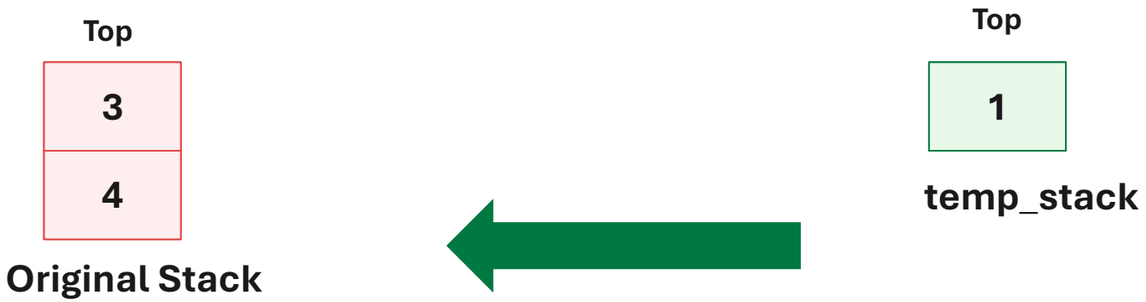
        # Insert temp into its correct position
        temp_stack.push(temp)
```

Pop 2. Since 4 > 2, move 4 from temp_stack back to the original stack.

Content Copyright Nanyang Technological University

3. Step 7 – Move 3 Back

Temp = 2




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

Compare again: temp_stack.peek() = 3 > 2, so move 3 back to the original stack.

Content Copyright Nanyang Technological University

3. Step 8 – Push 2 to temp_stack

Temp = 2




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

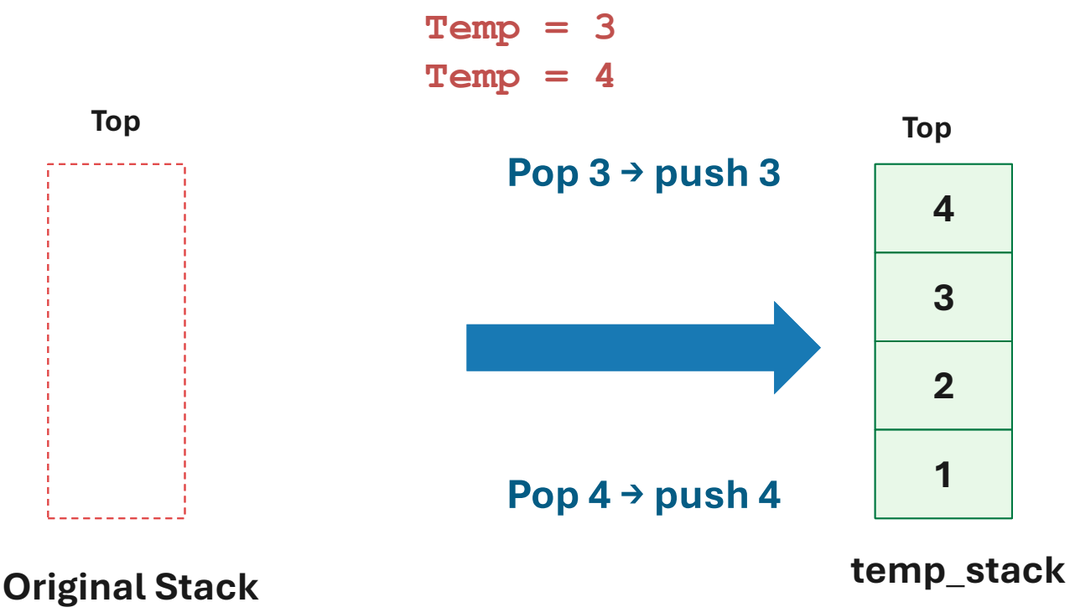
        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

Now temp_stack.peek() = 1 < 2, so push 2. temp_stack = [2,1].

Content Copyright Nanyang Technological University

3. Step 9 – Process Remaining 3 and 4




```
# Process each item from the original stack
    while not stack.is_empty():
        temp = stack.pop()

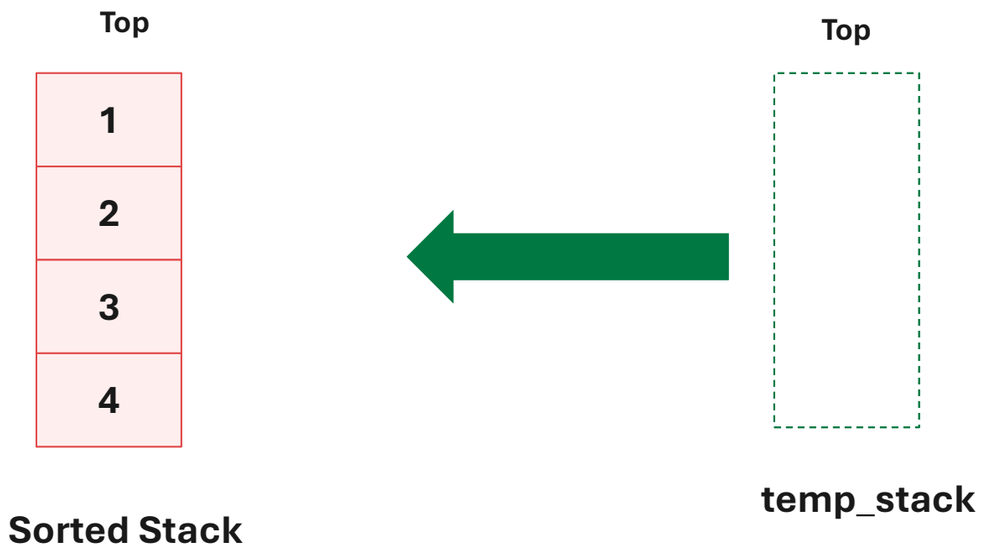
        # Move larger items back to the original stack
        # until the correct position for temp is found
        while not temp_stack.is_empty() and temp_stack.peek() > temp:
            stack.push(temp_stack.pop())

        # Insert temp into its correct position
        temp_stack.push(temp)
```

After processing the remaining items, temp_stack = [4,3,2,1] (top = 4).

Content Copyright Nanyang Technological University

3. Final Result – Move Back to Original Stack




```
# Move the sorted items back to the original stack
    while not temp_stack.is_empty():
        stack.push(temp_stack.pop())
```

Final sorted stack (top → bottom): 1, 2, 3, 4

Move 4, 3, 2, and 1 from temp_stack back to the original stack.

Content Copyright Nanyang Technological University